

mini_code

模块化 Python 编码代理系统

- mini_code: 模块化 Python 编码代理系统
- 核心能力: CLI 交互、模型调用、工具执行、任务管理、团队协作、上下文压缩
- 汇报重点: 系统设计 + 模型使用方式 + 模型能力分析

CLI

LLM

Team

Tools

State

模型能力

代码理解
规划生成

真实任务

文件 / 命令
状态 / 上下文

流程痛点

单轮问答
难以闭环

运行框架

可控 / 可扩展
可验证

- 大模型具备代码理解、规划和生成能力
- 真实开发任务需要：读写文件、执行命令、跟踪任务、长期上下文
- 单轮问答难以支撑复杂开发流程
- 需要一个可控、可扩展、可验证的代理运行框架

工程

清晰模块边界
便于扩展和测试

使用

CLI 完成代理交互
封装消息历史

协作

任务板 / 子代理
队友线程

安全

路径限制
危险命令拦截

研究

可观察调用
分析工具选择

基础交互

mini-code chat

工具能力

bash / 读写 / 编辑

状态能力

Todo / 任务板 / 会话

智能能力

工具调用 / 子代理 / 压缩

协作能力

队友 / inbox / 认领

配置能力

.env / 模型 ID / Base URL

概要设计

05

cli

命令行入口

runtime

代理主循环和工具调度

features / tools 任务、团队、文件、shell、子代理

llm / config

模型调用、日志、环境配置

状态目录

.tasks

.team

.sessions

.transcripts

.logs

- 架构分层: cli、runtime、features/tools、llm/config
- 依赖方向: CLI -> Runtime -> Features/Tools/LLM
- 状态目录: .tasks、.team、.sessions、.transcripts、.logs

1

压缩旧工具结果

2

注入后台结果和团队 inbox

3

构建系统提示词和工具 schema

4

调用模型

5

执行模型请求的工具

6

工具结果回注入历史

模型推理 → 工具执行 → 结果反馈 → 继续推理

```
AnthropicProvider  
.create_message()
```

统一入口 · 日志记录 · 可替换实现

输入

```
system prompt  
conversation history  
tool schemas  
max_tokens
```

输出

```
text 直接回复  
tool_use 触发真实工具执行
```

- 结构化日志: 记录 request、response、error、duration、usage

已利用能力



能力边界

- 能力边界：工具执行必须由系统兜底
- 上下文压缩可能丢失细节
- 复杂协作依赖明确协议
- 模型错误需通过测试和日志发现

模型适合负责理解、规划和决策；系统必须负责执行、安全和验证

- 单元测试覆盖配置、CLI、Runtime、LLM、工具、任务、团队、后台
- Fake Provider 模拟模型响应
- 验证 tool_use、压缩、日志、inbox、任务依赖

```
pytest -q
```

52 passed

in 0.28s

应用效果与总结

1

形成可运行的 mini 编码代理

2

支持本地工作区状态持久化

3

支持复杂任务拆分和多代理协作

4

支持模型调用可观测与可分析

模型：理解 · 规划 · 决策

系统：工具 · 安全 · 状态 · 验证

二者结合，才构成可靠编码代理